

The Invoice Pile Problem

Indian SMEs manually key hundreds of vendor bills every month — at a cost their accountants pay every day

COST OF MANUAL ENTRY

- › **8-12 min per invoice** × hundreds of bills per month
- › **3-5% error rate** — surfaces only at month-end reconciliation
- › **Zero audit trail** — no record of who entered what, or when
- › Accountants doing **data entry instead of advisory work**

This affects every SME working with an external accounting firm. FinBridge is the fix.

WHAT BREAKS WHEN THIS FAILS

- › GST filings with wrong input-tax-credit amounts
- › Payments sent to the wrong vendor account
- › Salary disbursements booked under the wrong cost centre
- › MIS reports delivered late — underlying data isn't clean

01

PROBLEM

The Invoice Pile Problem

FinBridge

A SaaS workspace shared between accounting firms and their clients — upload to approval in under 60 seconds

① Upload

Company user uploads any document — vendor invoice (PDF or image), salary register, bank statement. Any format, any Indian vendor.

② Extract

Claude vision or Gemini extracts vendor name, GSTIN, invoice #, date, CGST/SGST/IGST, and line items in 2–5 seconds.

Per-field confidence scores tell accountants exactly which values to double-check.

③ Review & Approve

Accountant sees the original document and extracted fields side-by-side with **color-coded confidence badges**.

One click to approve. Full **audit trail** written automatically.

- **Multi-company:** one accountant, all client companies in a single queue
- **Offline-capable:** fixture provider — 10 real Indian invoice PDFs bundled
- **Three-tier tenancy:** Platform → Firm → Company, service-layer isolation
- **Full audit trail:** every change logged with user + timestamp, shown in UI

02

SOLUTION

From Pile to Ledger
in Seconds

Upload to Approval in Under 60 Seconds

Company User → AI Extraction → Accountant Review → Accepted Ledger

1

Company User — Upload

- > PDF, image, or Excel
- > Any Indian vendor format
- > MIME-validated, 10 MB cap
- > Stored with randomised name

2

AI — Extract

- > Vendor name + GSTIN
- > Invoice #, date, total
- > CGST / SGST / IGST
- > Line items + payment head
- > Confidence % per field **NEW**

3

Accountant — Review

- > Doc + fields side-by-side
- > Confidence badges **NEW**
- > Editable extracted fields
- > Audit trail inline **NEW**

4

Outcome

- > Approve → accepted ledger
- > Audit log auto-written
- > Reject → with note
- > Request info → re-submit

Fixture provider matches filenames to pre-built JSON — 10 real Indian PDFs in `sample_invoices/`. Claude & Gemini fully implemented; one env var to switch.

03

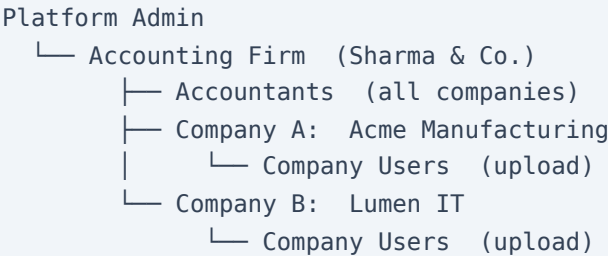
DEMO

Upload to Approval
Under 60 Seconds

One Stack, Three Tiers, Three AI Providers

Single public port · Application-layer tenant isolation · Pluggable AI

TENANCY MODEL



Every DB row scoped by `firm_id` + `company_id` .
Single `tenant_scope()` FastAPI dependency enforces isolation — verified by a cross-tenant smoke test on every build.

DEPLOYMENT

```
nginx :80 (public)
├─ proxy_pass → app :8000 (internal)
│   ├─ /api/* FastAPI routes
│   └─ /* React 18 SPA
postgres :5432 (internal only)
```

AI EXTRACTION PROVIDERS

Provider	When to use	Model
fixture	Demo / CI / offline	Pre-built JSON
claude	Production primary	claude-sonnet-4-6
gemini	Fallback / cost-sensitive	gemini-2.5-flash

All three implement `ExtractionProvider` and return `ExtractedInvoice` with per-field confidence scores.
Switch providers with one env var — no code change.

TECH STACK

- Python 3.12 · FastAPI 0.115 · SQLAlchemy 2.0 + Alembic · Postgres 16 · psycopg3
- React 18 · TypeScript · Vite · Tailwind CSS · shadcn/ui · TanStack Query v5 · Zustand v4
- Docker Compose · nginx · multi-stage build · idempotent seed script

48 Hours → Production-Ready Demo

~4,800

Lines of Python

~5,500

TypeScript / TSX

3

AI Providers (all live)

10

Indian Invoice PDFs

BACKEND

- › JWT auth · 4-role RBAC (platform, firm, accountant, company)
- › Three-tier tenant isolation via single FastAPI dependency
- › Fixture + Claude + Gemini — **all three fully implemented**
- › 7 API modules incl. new **audit log API** (`GET /api/audit`)
- › UUID-aware fixture matching for stored uploads
- › Alembic migrations · 8 ORM models

FRONTEND + INFRASTRUCTURE

- › 12 pages across 4 roles · all wired to real API hooks
- › **Per-field AI confidence badges** — green / yellow / red
- › **Audit trail panel** in the transaction detail page
- › Blob-based PDF/image rendering (auth-correct in iframe)
- › nginx on :80 · 11 MB upload limit · 120s timeouts
- › **E2E test suite**: all 10 PDFs through approve / reject / needs-info + auth guards

05

WHAT WE BUILT

48 Hours →
Production-Ready
Demo

Where FinBridge Goes Next

From hackathon demo to production accounting platform

Near-term

- › Bank statement reconciliation — match payments to invoices, flag unmatched debits
- › Bulk upload with parallel extraction queue
- › Live AI provider toggle in the UI

Mid-term

- › ERP export: Tally, Zoho Books, QuickBooks
- › Email notifications on approval / rejection
- › GST return pre-fill — GSTR-1 / GSTR-2B reconciliation

Infrastructure

- › S3-compatible object storage (one provider swap)
- › Postgres row-level security — second isolation layer
- › Refresh token rotation + MFA

06

ROADMAP

Where FinBridge
Goes Next

Built in 48 hours · One developer + Claude Code · Python + React + PostgreSQL · `docker compose up --build`

Developing Alongside Claude Code

Pair programming at 10× speed — what worked, what didn't, and what stayed human

What Claude Code built

- › Full backend scaffold — FastAPI app factory, SQLAlchemy models, Alembic migrations, JWT auth, all API routes and services
- › Three AI extraction providers — including a fully working Gemini implementation from scratch
- › Complete React frontend — 12 pages, all TanStack Query hooks, Zustand auth store, protected routes
- › Docker multi-stage build, nginx config, seed script, 8 fixture JSONs for Indian invoices
- › E2E test suite covering all 10 sample PDFs across approve / reject / needs-info / auth-guard paths
- › This presentation deck and demo script

The human-AI dynamic

- › **PLAN.md as shared context** — Claude Code followed the build sequence without losing coherence across 9,000+ lines
- › **Caught real bugs** — wrong psycopg DSN format, bcrypt version conflict, UUID-prefixed filename fixture matching
- › **Every diff reviewed before committing** — no blind "vibe coding"; architecture decisions stayed human-driven
- › **Fast iteration on feedback** — "font is too small, lots of empty space" → fixed layout in one turn
- › **What stayed human**: trade-off decisions, cut order, the demo narrative, knowing when "good enough" ships

48 hours · 1 developer · 1 AI pair programmer · ~9,500 lines of working, tested code

07

EXPERIENCE

Developing
Alongside Claude
Code